



FACE RECOGNITION-BASED LOGIN SYSTEM USING JAVA AND OPENCV



INTRODUCTION

In the modern world, where data security and user authentication are of utmost importance, biometric technologies have emerged as a reliable and efficient alternative to traditional password-based systems. Among all biometric methods, face recognition stands out due to its non-intrusive nature, ease of implementation, and growing support in modern computing libraries.

This project demonstrates a **Face Recognition-Based Login System** developed entirely in ***Java**, **integrated** with **OpenCV** for face detection, and **JavaFX** for GUI. Unlike conventional approaches that use **LBPH (Local Binary Pattern Histograms)** or deep learning models, this system uses a simplified pixel-difference-based approach to recognize faces in real-time without requiring any internet or heavy ML training.



OBJECTIVE

THE MAIN OBJECTIVES OF FACE RECOGNITION ARE:

- ✓ Build a user-friendly face login system using Java.
- ✓ Enable face detection using OpenCV Haar cascades.

- ✓ Store user face data securely for future comparisons.
- ✓ Allow real-time matching of face data to validate user identity.
- ✓ Avoid use of complex models like LBPH for lightweight implementation.
- ✓ Provide modular code structure for future expansion.

THE GOAL OF THE ONSLAUGHT GAME IS TO:

- 🧠 Test the reflexes and aiming accuracy of players.
- 🔪 Survive increasingly difficult enemy waves.
- 👤 Defend your base and upgrade your weapons.
- ✂ Provide a retro, arcade-like 2D shooter experience.

✂ TOOLS AND TECHNOLOGIES USED

THIS SYSTEM WAS IMPLEMENTED USING THE FOLLOWING TOOLS AND TECHNOLOGIES:

- 💻 **Programming Language:** Java (JDK 17+)
- 🔧 **GUI Library:** JavaFX for interface and user navigation
- 🖼 **Image Processing Library:** OpenCV (version 4.8.0)
- 📁 **Build Tool:** Apache Ant (for compiling OpenCV Java bindings)

🖼️ **Face Detection:** Haar Cascade Classifiers

💾 **Data Storage:** Image files stored in folder structure

🖨️ **Platform:** Windows OS

🧠 SYSTEM ARCHITECTURE

THE PROJECT FOLLOWS A MODULAR ARCHITECTURE WITH EACH FUNCTIONALITY ISOLATED INTO COMPONENTS:

- ➦ **MAIN.JAVA:** Entry point of the application that links different screens.
- ➦ **FACECAM.JAVA:** Captures live video feed from the webcam and detects faces using Haar cascades.
- ➦ **FACEREGISTRATION.JAVA:** Captures and stores grayscale cropped face images for new users.
- ➦ **FACELOGIN.JAVA:** Matches captured live face with registered face data using image difference.
- ➦ **FOLDER STRUCTURE:** Each user has a separate folder under `registered_faces/` where face images are stored.

THIS ARCHITECTURE MAKES IT EASY TO DEBUG, UPGRADE, AND EXTEND THE FUNCTIONALITY LATER (E.G., BY ADDING DNN SUPPORT).

🔄 WORKING MECHANISM

THE SYSTEM WORKFLOW IS AS FOLLOWS:

STEP 1: USER REGISTRATION

- ✓ User launches the app and enters a username.
- ✓ The webcam is activated, and face detection begins.
- ✓ Once a face is detected, multiple grayscale images (face crops) are captured stored under the user's folder.

STEP 2: LIVE FACE DETECTION

- ✓ During login, webcam feed is activated again.
- ✓ Haar cascade detects the user's face in real-time.
- ✓ The face image is cropped and converted to grayscale for comparison.

STEP 3: FACE MATCHING

- ✓ System compares the current face with stored images using pixel-wise absolute difference.
- ✓ If the cumulative difference (sum of all pixel intensity differences) is below a set threshold, the user is considered authenticated.

STEP 4: LOGIN DECISION

- ✓ If matching is successful: "Access Granted".

✓ Otherwise: "Face not recognized. Access Denied."

FEATURES

THIS FACE LOGIN SYSTEM OFFERS THE FOLLOWING PRACTICAL FEATURES:

- 🖥️ **REAL-TIME FACE DETECTION:** Uses OpenCV Haar classifiers.
- 📁 **OFFLINE OPERATION:** No internet or online APIs needed.
- 👤 **MULTI-IMAGE REGISTRATION:** Allows storing multiple face images.
- 💡 **LIGHTWEIGHT IMPLEMENTATION:** Suitable for devices with limited resources.
- 🎨 **JAVAFX UI:** Simple, clean interface for easy usage.
- 🧩 **MODULAR CODEBASE:** Easy to extend and maintain.

✓ ADVANTAGES

IMPLEMENTING A FACE LOGIN SYSTEM USING THIS APPROACH COMES
WITH SEVERAL BENEFITS:

- 🔒 **SECURE LOGIN:** No password is stored or used.
- 🧠 **SIMPLE LOGIC:** No external model training is needed.
- 🛠️ **DEVELOPER FRIENDLY:** Ideal for students and developers for image processing.
- 📦 **PORTABLE:** Can be deployed on any machine with a webcam and Java installed.

LIMITATIONS

HOWEVER, AS THIS SYSTEM AVOIDS USING ADVANCED MACHINE LEARNING TECHNIQUES, IT DOES HAVE LIMITATIONS:

-  **LOWER ACCURACY:** Not suitable for production-grade systems.
-  **LIGHTING SENSITIVITY:** Accuracy drops in low-light environments.
-  **NO ROTATION INVARIANCE:** Only works with frontal face images.
-  **FALSE POSITIVES:** May incorrectly match similar faces.

FUTURE ENHANCEMENTS

TO MAKE THIS SYSTEM MORE ROBUST AND PRODUCTION-READY, THE FOLLOWING UPGRADES ARE RECOMMENDED:

-  **USE LBPH/DNN/CNN:** For more reliable and accurate face recognition.
-  **LIVENESS DETECTION:** Prevent spoofing using photos or videos.
-  **LOGGING SYSTEM:** Track login attempts and failures.
-  **DATABASE INTEGRATION:** Use SQLite or MySQL to store metadata securely.
-  **MULTI-PLATFORM SUPPORT:** Port to mobile and web platforms.
-  **MULTI-USER ROLE MANAGEMENT:** Differentiate between admin and user.



INTEGRATING ONSLAUGHT: A 2D SHOOTING GAME

In addition to the biometric login system, this project features a **Java-based 2D shooting game** called **Onslaught**. Once a user logs in successfully via the face recognition system, they can access and play the Onslaught game directly from the dashboard. This makes the project more interactive, secure, and exciting for end users, especially in gaming or academic environments.



GAME ARCHITECTURE

THE GAME WAS BUILT USING:



LANGUAGE: Java



FRAMEWORK: LibGDX (for game loop and rendering)



GUI INTEGRATION: JavaFX (launch + post-login menu)



GAMEPLAY LOGIC: Java classes for player, enemies, bullets, collisions



PHYSICS: Custom collision detection engine



AUDIO: JavaFX Media Player for background music and sound effects



KEY FEATURES



ENEMY WAVES

Enemies spawn in increasing waves with rising difficulty.

WEAPON SYSTEM

Multiple weapons: pistol, shotgun, sniper, with reload mechanics and ammo count.

FACE-BASED GAME ACCESS

Only authenticated users (via face login) can access and play the game.

CUSTOM UI

Game menus, health bar, score display, and weapon HUD using JavaFX/LibGDX.

SCORE TRACKING

Game keeps track of enemies killed, accuracy, survival time, and high scores.

INTEGRATION WITH FACE LOGIN

ONCE A USER PASSES THE *FACE AUTHENTICATION*, THE SYSTEM
AUTO-LAUNCHES THE ONSLAUGHT GAME:

```
if (faceMatchSuccessful) {  
    GameLauncher.launchGame();  
}
```

THIS ENSURES:

AUTHORIZED GAME ACCESS ONLY

SEAMLESS USER EXPERIENCE

SECURITY + FUN COMBO



UPCOMING UPDATES



Integrate neural AI enemies.



Add online multiplayer with socket programming.



Store player stats in cloud DB.



Add customizable avatars, skins, and power-ups.



Use leaderboard system with authenticated users only.



SAMPLE CODE SNIPPET – FACE MATCHING

```
public boolean isFaceMatching(Mat inputImage, File[] registeredImages) {
    for (File imageFile : registeredImages) {
        Mat registered = Imgcodecs.imread(imageFile.getAbsolutePath(),
            Imgcodecs.IMREAD_GRAYSCALE);
        Imgproc.resize(inputImage, inputImage, new Size(100, 100));
        Imgproc.resize(registered, registered, new Size(100, 100));

        Core.absdiff(inputImage, registered, diff);
        Scalar sumDiff = Core.sumElems(diff);

        if (sumDiff.val[0] < 2000000) {
            return true;
        }
    }
    return false;
}
```

THIS LOGIC CHECKS IF THE LIVE IMAGE IS SIMILAR ENOUGH TO ANY REGISTERED IMAGE BY COMPARING PIXEL DIFFERENCES.

FOLDER STRUCTURE

ONSLAUGHT_FACE_RECOGNITION/

- Main.java
- FaceCam.java
- FaceRegistration.java
- FaceLogin.java
- assets/
 - haarcascade_frontalface_alt.xml
- registered_faces/
 - [username]/
 - face1.png
 - face2.png
- opencv/
 - opencv_java480.dll

ONSLAUGHT_GAME/

- GameLauncher.java
- GameScreen.java
- Player.java
- Enemy.java
- Bullet.java
- Assets/
 - sprites/
 - sounds/
 - fonts/
- utils/
 - CollisionDetector.java



EDUCATIONAL VALUE

THIS COMBINED PROJECT IS IDEAL FOR:

🎓 Engineering Students (CSE, IT, AI, Game Dev)

🔧 Mini/Major Projects in Semester VI–VIII

🏢 Internship Projects with biometric + gaming modules

👨🏫 Teaching Assistants/Demo Projects in colleges

📌 CONCLUSION

This project successfully demonstrates how a lightweight and functional **face recognition login system** can be developed using Java and OpenCV without relying on complex machine learning models. Though it lacks the sophistication of professional-grade systems, it offers a practical solution for offline authentication and serves as a valuable learning tool for students and hobbyists.

Its modular nature and clear separation of concerns make it ideal for enhancements, such as integration of LBPH, DNN, or database storage. The foundation laid by this project can be expanded into more sophisticated biometric authentication systems in future developments.

Together, the **face recognition login system** and the **Onslaught 2D shooting game** present a unique fusion of **biometric security** and **interactive** entertainment. This combination shows how Java can be used not just for backend or enterprise-level development, but also for real-time applications **involving image processing, graphics, and game logic** — all in one integrated system.



REFERENCES

- OpenCV Documentation – [OpenCV](#)
- JavaFX Official Guide – [OpenJFx](#)
- Haar Cascade XML – [OpenCV Haar Classifiers GitHub](#)
- Java Tutorials – [Oracle JAVA 8](#)



SCREENSHOTS



